Student Management System - EduPortal

A comprehensive, modern Student Management Portal built with a Node.js/Express backend and a responsive Vanilla HTML/CSS/JS frontend. It supports two primary roles: **Admin** and **Student**, handling everything from academic material distribution and announcements to support tickets and two-factor authentication.

🚀 Tech Stack

- **Backend:** Node.js, Express.js
- **Database:** PostgreSQL (Neon Serverless Postgres)
- **Frontend:** Vanilla HTML, CSS (Custom styling), JavaScript
- **Authentication:** JWT (JSON Web Tokens) & 2FA (Speakeasy / QR Code)
- **File Uploads:** Multer (Local storage in `/uploads`)

🔑 Key Features

Admin Portal

- **Dashboard Analytics:** Real-time metrics for total/active students, unread messages, announcements, and e-content materials.
- **Student Management:** Add new students, edit details via modal, and bulk-delete multiple students at once.
- **Announcements Engine:** Publish global announcements to all students. Tracks read receipts.
- **E-Content Library:** Upload academic materials (PDFs, Images, Videos, Audio). Automatically secures downloads to authenticated students.
- **Support Ticket Desk:** Manage and respond to student-raised IT/Academic support tickets (Pending, In Progress, Completed).
- **Direct Messaging:** Respond directly to student queries via the unified portal inbox.

Student Portal

- **Secure Authentication:** JWT-based login with mandatory **Two-Factor Authentication (2FA)** setup via QR Code (Google/Microsoft Authenticator).
- **Personal Dashboard:** Tracks personal activity, recent announcements, and quick-access metrics.
- **Support Ticket System:** Raise detailed support tickets with priority levels, categories, and file attachments.
- **Unified Inbox:** A centralized inbox for system notifications, ticket status updates, and direct admin replies. Read receipts automatically clear notification badges.
- **Material Access:** View and securely download E-content materials uploaded by the administration.
- **Responsive UI:** Fully responsive design with a collapsible sidebar toggle for mobile, tablet, and desktop viewing.

📁 Project Structure

```
├── backend/
│   ├── config/          # Database connection (pg-pool) and initialization
│   ├── controllers/     # Business logic for auth, students, tickets, announcements, etc.
│   ├── middleware/      # JWT verification, Admin/Student role checks, Multer uploads
│   ├── routes/          # Express API endpoints
│   └── server.js        # Main Node.js server entry point
├── database/
│   └── schema.postgres.sql # Complete PostgreSQL schema definition
├── frontend/
```

```
│   ├── admin/          # Admin HTML views
│   ├── student/        # Student HTML views
│   ├── css/            # Global stylesheets and responsive layout
│   └── js/             # Client-side API interactions and UI logic
└── uploads/            # Directory for E-Content and Ticket attachments
```

🛠️ Setup & Installation

1. **Install Dependencies:** Run the following command in the root directory:

   ```
   npm install
   ```

2. **Database Configuration:** The application uses a PostgreSQL database. Ensure your connection string (like a Neon DB URL) is injected into the environment as `DATABASE_URL`. The application will automatically run `database/schema.postgres.sql` on startup to generate missing tables.

3. **Start the Server:**

   ```
   npm start
   ```

   The backend will boot up on the configured port (default `3000`), serving the frontend static files simultaneously.

4. **Access the Application:** Open your browser and navigate to `http://localhost:3000`.

🔄 Recent System Updates

- **Database Migration:** Successfully migrated schema to PostgreSQL, removing old SQLite remnants.
- **Ticket System Fix:** Injected the previously missing `tickets` table and corrected the automated inbox mailing constraints.
- **Responsive Layout:** Fixed mobile stacking, restored modal alignments, and implemented a universal sidebar hide/toggle button.
- **Notification Syncing:** Synchronized "Unread" badges for Announcements and Inbox to clear accurately upon viewing.